

Lista de Exercícios – MiniCCO-0

Núcleo Geométrico e Estrutural para Crescimento Arterial Simplificado

Disciplina: Algoritmos e Programação Avançada
Prof. Dr. Rafael Alves Bonfim de Queiroz

2026.1

1 Objetivo

O objetivo desta atividade é implementar, em linguagem C ou C++, um núcleo inicial simplificado inspirado no método CCO (*Constrained Constructive Optimization*). Este núcleo não corresponde ao método completo, mas deve fornecer os principais blocos fundamentais que serão reutilizados no trabalho prático final da disciplina.

O aluno deverá desenvolver:

- Representação de pontos, segmentos e nós arteriais;
- Estrutura de árvore binária simples;
- Crescimento recursivo da árvore;
- Testes geométricos fundamentais;
- Função de custo geométrica simples;
- Seleção de conexões baseada em ordenação.

2 Descrição do Problema

Implemente um programa que construa uma árvore arterial simplificada em 2D dentro de um domínio circular.

O algoritmo deve:

1. Iniciar com um nó raiz;
2. Gerar iterativamente pontos terminais aleatórios dentro do domínio;
3. Para cada novo ponto:
 - Gerar conexões candidatas com a árvore existente;
 - Validar restrições geométricas;

- Calcular um custo geométrico;
- Selecionar a melhor conexão válida;
- Inserir o novo nó na árvore.

3 Especificação Técnica

3.1 Entradas

O programa deve receber via linha de comando:

- N_{term} : número de pontos terminais;
- R : raio do domínio circular.

Exemplo:

```
./programa 100 10.0
```

3.2 Saídas

O programa deve:

- Construir a árvore arterial;
- Imprimir:
 - Número total de nós;
 - Número de folhas;
 - Comprimento total da árvore;
 - Número de conexões rejeitadas;
- Exportar um arquivo com os segmentos (formato .txt, .csv ou .vtk).

4 Estruturas de Dados

Sugere-se a implementação das seguintes estruturas:

```
typedef struct {
    double x, y;
} Point;

typedef struct {
    Point a, b;
} Segment;

typedef struct No {
    struct No *esq;
    struct No *dir;
    struct No *pai;
    Point p;
    int id;
} No;

typedef No* ptrNo;
```

5 Funções a Implementar

5.1 Geometria

- Distância entre pontos:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Orientação:

$$\text{orient}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x)$$

- Interseção de segmentos;
- Distância ponto-segmento;
- Teste de domínio:

$$x^2 + y^2 \leq R^2$$

5.2 Estrutura e Algoritmos

- Inserção de nós na árvore;
- Percurso recursivo da árvore;
- Geração de candidatos;
- Ordenação de candidatos por custo;

6 Função Custo

A função custo é responsável por avaliar a qualidade de uma conexão candidata entre um novo ponto terminal e a árvore existente. O objetivo é selecionar a conexão que minimiza um critério geométrico simples, representando uma aproximação inicial do comportamento do método CCO.

6.1 Caso básico (obrigatório)

A função custo mais simples é baseada apenas na distância euclidiana:

$$J = d(P_{\text{novo}}, P_{\text{conexao}})$$

onde:

- P_{novo} : ponto terminal gerado aleatoriamente;
- P_{conexao} : ponto da árvore onde a conexão será feita;
- $d(\cdot, \cdot)$: distância euclidiana.

A distância é dada por:

$$d(P, Q) = \sqrt{(x_Q - x_P)^2 + (y_Q - y_P)^2}$$

Interpretação: esta função favorece conexões mais curtas, levando à construção de árvores mais compactas.

6.2 Caso estendido (opcional)

Uma versão mais sofisticada da função custo inclui também uma penalização angular:

$$J = \alpha \cdot d + \beta \cdot \theta$$

onde:

- $\alpha, \beta > 0$: pesos definidos pelo usuário;
- d : distância entre os pontos;
- θ : ângulo entre o novo segmento e o segmento existente.

6.3 Cálculo do ângulo

Considere:

- vetor existente: $\vec{u} = B - A$
- novo vetor: $\vec{v} = P_{novo} - P_{conexao}$

O ângulo θ é dado por:

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}$$
$$\theta = \arccos\left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}\right)$$

Produto escalar:

$$\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y$$

6.4 Interpretação geométrica

- A componente d favorece conexões curtas;
- A componente θ favorece bifurcações mais suaves;
- Ângulos muito agudos ou muito obtusos são penalizados.

6.5 Normalização (recomendado)

Para evitar que uma componente domine a outra, pode-se normalizar:

$$J = \alpha \cdot \frac{d}{d_{max}} + \beta \cdot \frac{\theta}{\pi}$$

onde:

- d_{max} : maior distância observada entre candidatos;
- $\theta \in [0, \pi]$.

6.6 Valores sugeridos

- $\alpha = 1.0$
- $\beta = 0.5$

6.7 Observação importante

Neste exercício, a função custo é apenas uma aproximação simplificada. No método CCO completo, o custo envolve grandezas físicas como:

- volume da árvore;
- resistência ao fluxo (Lei de Poiseuille);
- pressão e fluxo.

Esses aspectos serão incorporados nas próximas etapas da disciplina.

7 Restrições

Uma conexão só pode ser aceita se:

- O ponto estiver dentro do domínio;
- Não houver interseção com outros segmentos;
- A distância mínima a outros segmentos for maior que ε .

8 Visualização (Python + PyVista)

O aluno deverá desenvolver um script em Python para visualizar a árvore gerada.

Sugestão:

```
import pyvista as pv
import numpy as np

points = []
lines = []

# carregar arquivo gerado pelo C/C++

mesh = pv.PolyData()
mesh.points = np.array(points)
mesh.lines = np.hstack(lines)

plotter = pv.Plotter()
plotter.add_mesh(mesh, line_width=3)
plotter.show()
```

9 Entregáveis

- Código-fonte completo (C ou C++);
- Script de visualização em Python;
- Relatório (2 a 4 páginas) contendo:
 - Descrição da implementação;
 - Exemplos de execução;
 - Análise de resultados;
 - Discussão sobre complexidade:
 - * Busca de candidatos;
 - * Teste de interseção;

10 Testes

Executar o programa com pelo menos três valores de N_{term} :

- Pequeno: 10
- Médio: 50
- Grande: 100 ou mais

11 Critérios de Avaliação

- Correção da implementação (30%)
- Estrutura de dados e organização do código (20%)
- Implementação geométrica correta (20%)
- Relatório e análise (15%)
- Visualização (PyVista) (15%)